

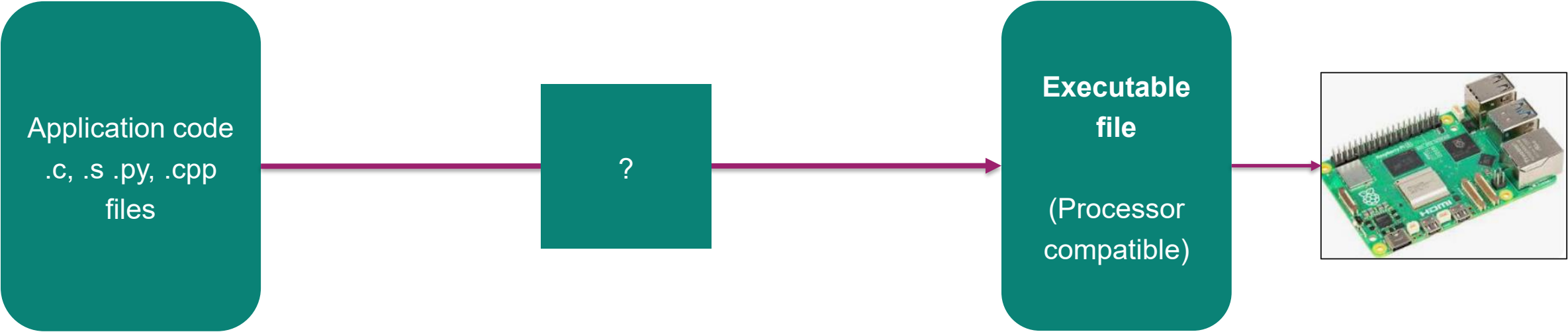


C program Segments, Journey from Source to binary

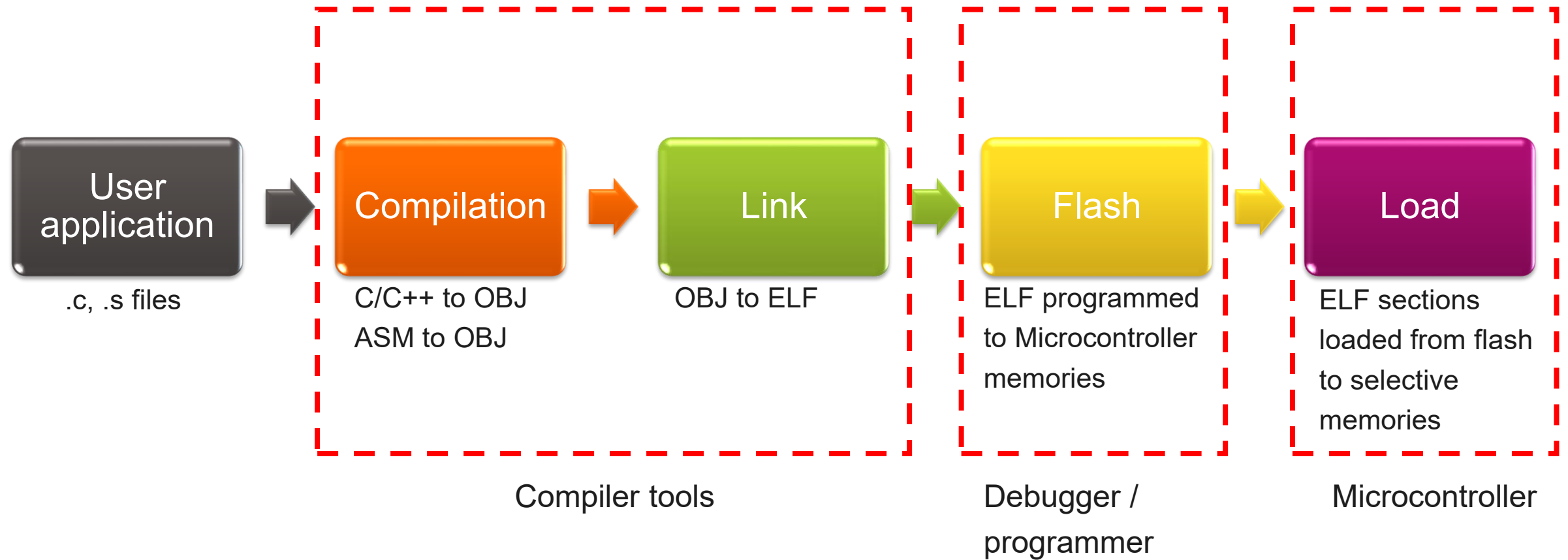
Boya Vinay Kumar
04 Oct 2025



Source to binary



Source to binary - Flow



Site of action



Host PC with cross-compiler for target CPU (e.g. ARM Cortex-M0)

Compilation, Linking and ELF generation



This debugger box helps in programming ELF sections of your application to Non-Volatile memories of the microcontroller

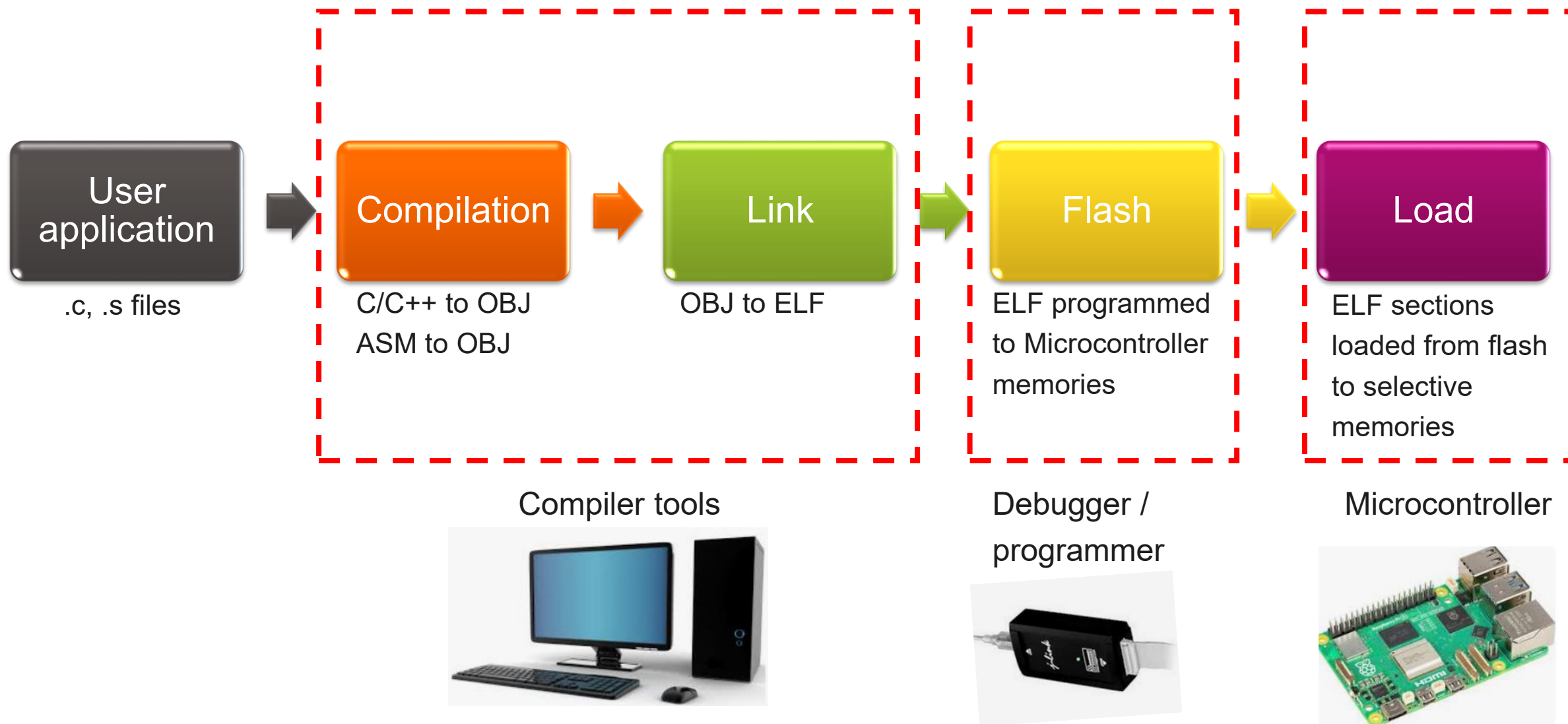


When you power-up the CPU

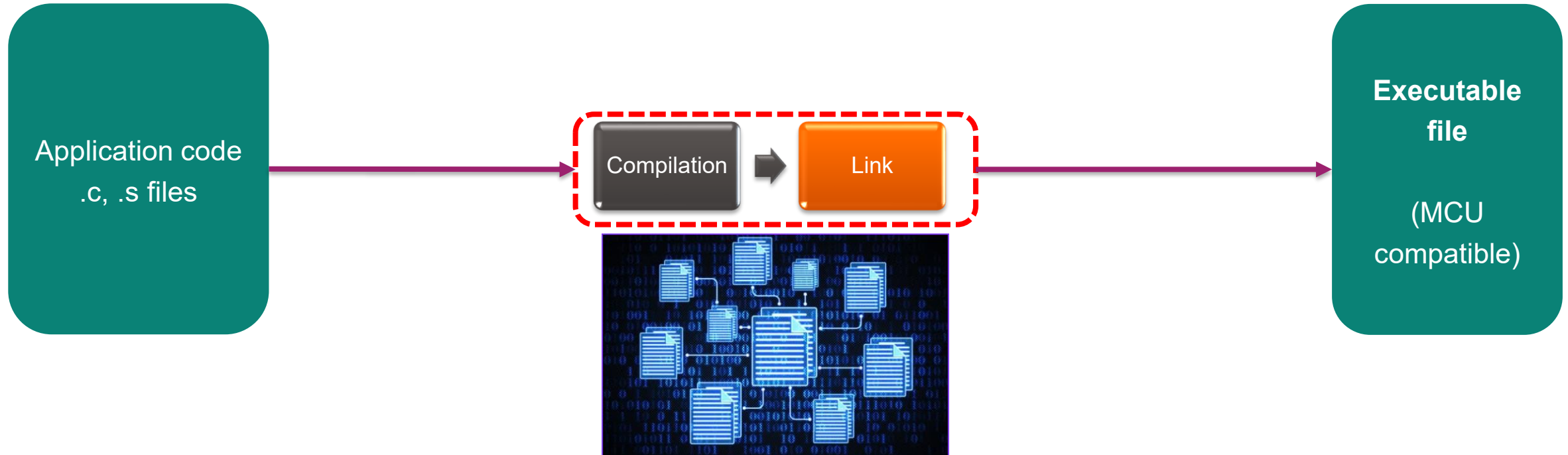
The startup software

1. Copies data sections of your application from flash to memory,
2. Clears BSS addresses
3. Optionally copies time critical program sections from flash to SRAM
4. Passes control to your main()

Source to binary - Flow



Compilation



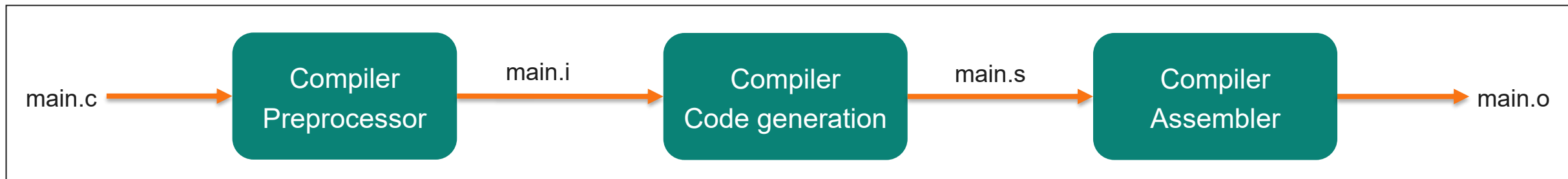
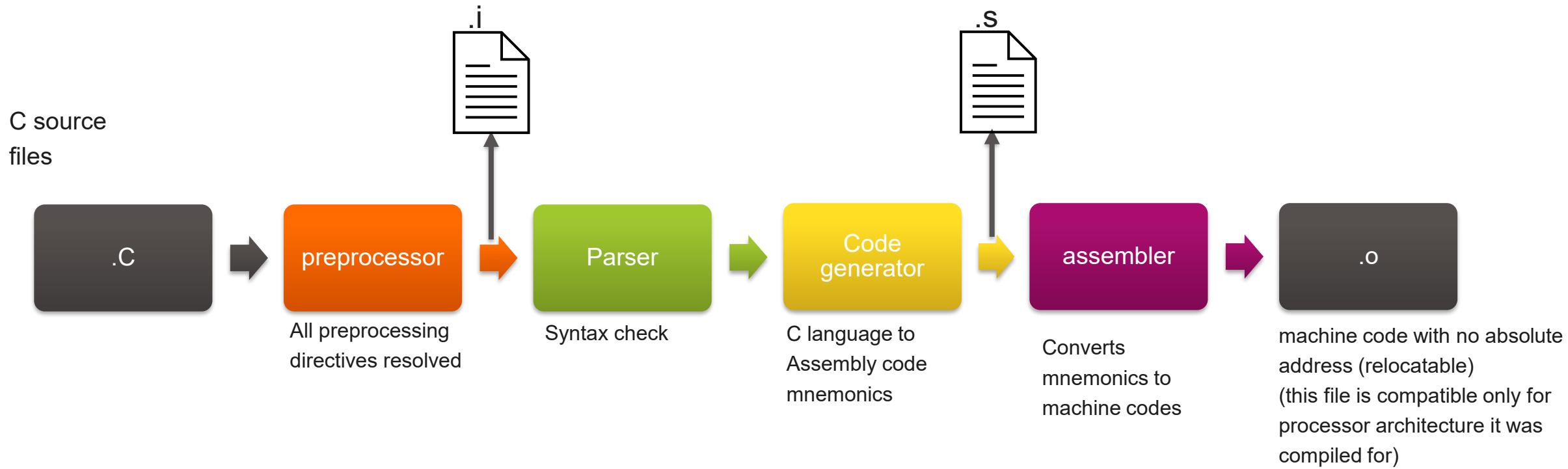
Cross compiler tool:

A collection of tools designed to transform source code into executable files.

Popular compilers:

1. GNU tools (GCC) for ARM Embedded Processors (free and open source)
2. Armcc from ARM Ltd. (ships with KEIL, license)

Source code compilation



Important commands for arm gcc tool chain

- arm-none-eabi-gcc
➔
Compiler, Linker, Assembler

<code>arm-none-eabi-gcc</code>	<code>-E source.c</code>	<code>-o destination.i</code>
<code>arm-none-eabi-gcc</code>	<code>-S source.i</code>	<code>-o destination.s /</code>
<code>arm-none-eabi-gcc</code>	<code>-S source.c</code>	<code>-o destination.s</code>
<code>arm-none-eabi-gcc</code>	<code>-c source.c</code>	<code>-o destination.o</code>

- arm-none-eabi-objdump
➔
ELF file analyzer

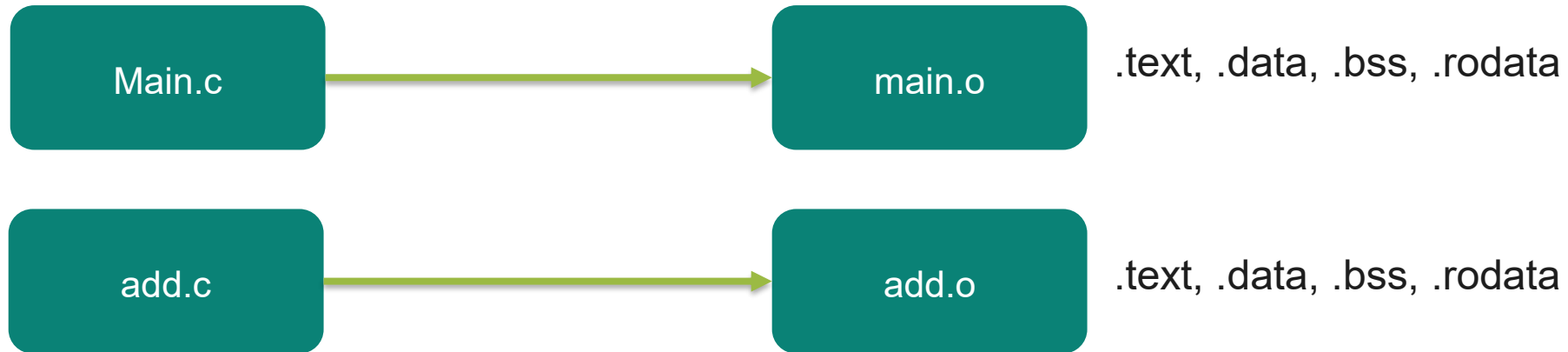
<code>arm-none-eabi-objdump</code>	<code>-h destination.o</code>
------------------------------------	-------------------------------

- arm-none-eabi-objcopy
➔
Format converter (.elf to .hex)

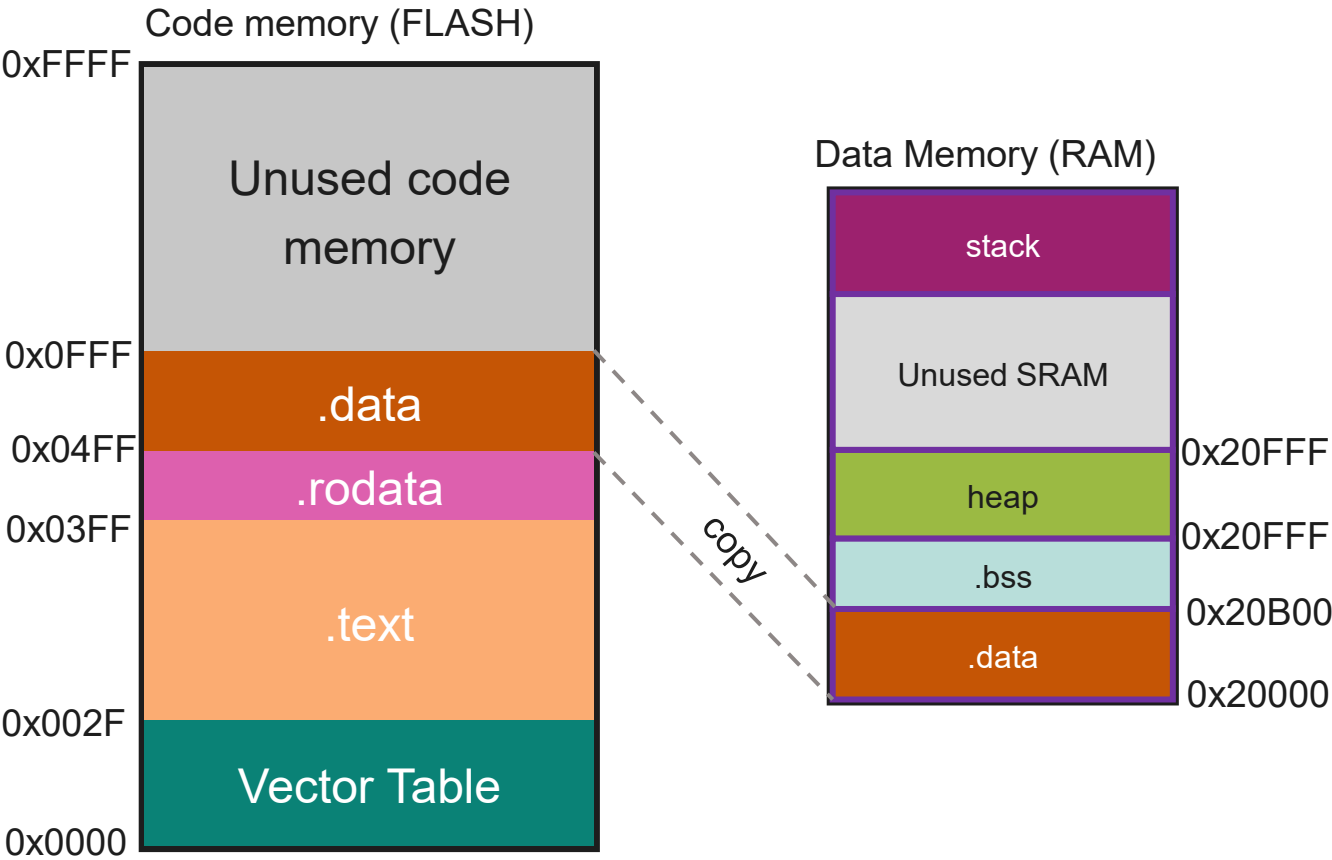
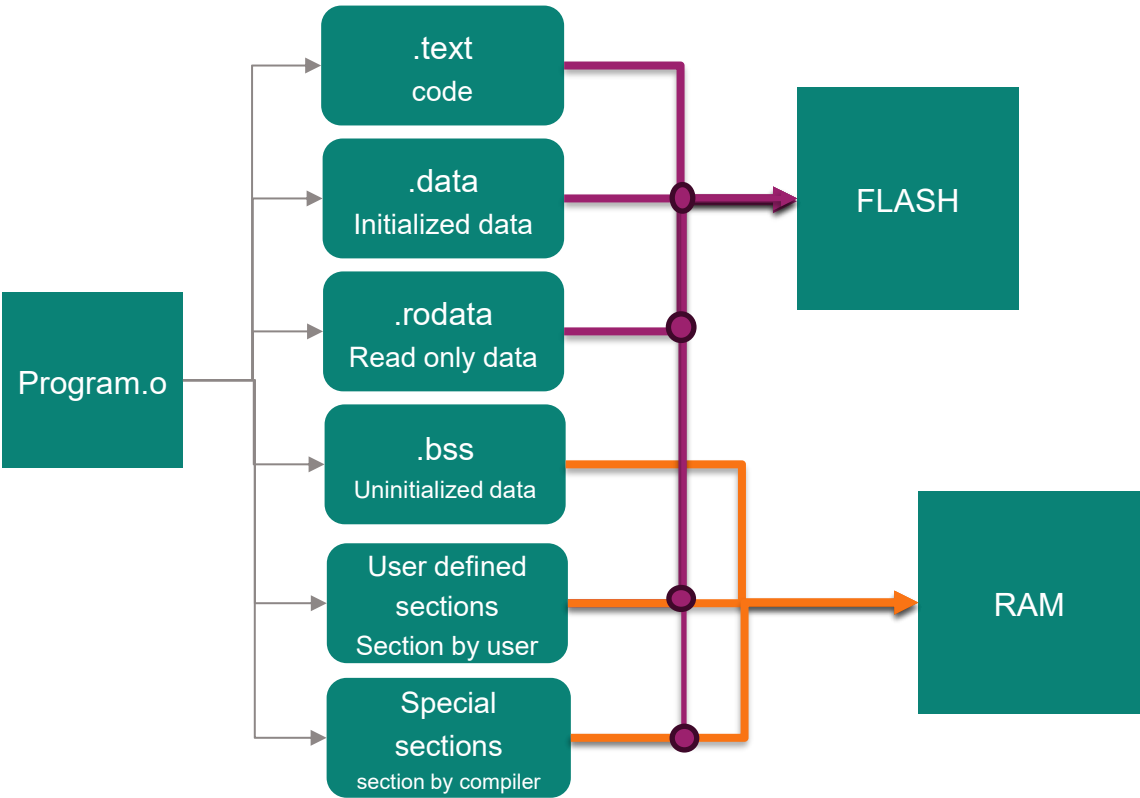
Below two commands are invoked by arm-none-eabi-gcc

- arm-none-eabi-as
➔
Assembler to convert .s to .o
- arm-none-eabi-ld
➔
Linker

Sections of object file



Sections and its memory!



Where does variables store in memory?

```
#include <stdio.h>

int global_init = 42;
int global_uninit;
static int global_static_init = 100;
static int global_static_uninit;

void example_function(void)
{
    int local_init = 10;
    int local_uninit;
    static int local_static_init = 50;
    static int local_static_uninit;

    // Modify static variables to show they retain values
    local_static_init++;
    local_static_uninit++;
}

int main(void) {
    // Call function twice to demonstrate static variable persistence
    example_function(); //First Call
    example_function(); // Second Call

    while(1);

    return 0;
}
```

Where does variables store in memory?

```
#include <stdio.h>

int global_init = 42;    // Initialized global variable (stored in data segment)
int global_uninit;       // Uninitialized global variable (stored in BSS segment, initialized to 0 by default)
static int global_static_init = 100; // Initialized global static variable (stored in data segment, file scope)
static int global_static_uninit;    // Uninitialized global static variable (stored in BSS segment, initialized to 0, file scope)

void example_function(void)
{
    int local_init = 10;    // Initialized local variable (stored on stack)
    int local_uninit;       // Uninitialized local variable (stored on stack, contains garbage value)
    static int local_static_init = 50; // Initialized local static variable (stored in data segment, retains value across calls)
    static int local_static_uninit; // Uninitialized local static variable (stored in BSS segment, initialized to 0, retains value across calls)

    // Modify static variables to show they retain values
    local_static_init++;
    local_static_uninit++;
}

int main(void) {
    // Call function twice to demonstrate static variable persistence
    example_function(); //First Call
    example_function(); // Second Call

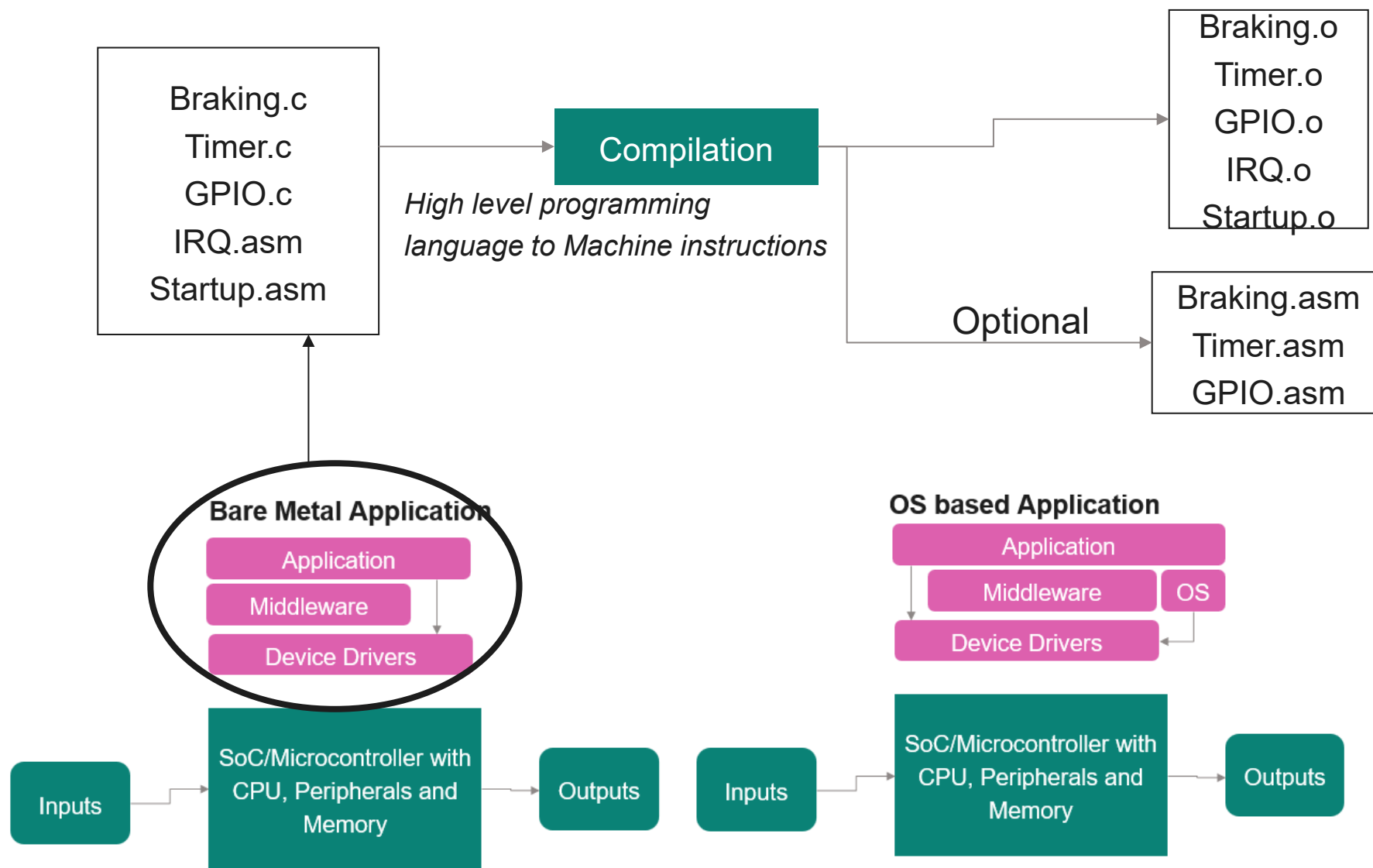
    while(1);

    return 0;
}
```

Sections and its memory!

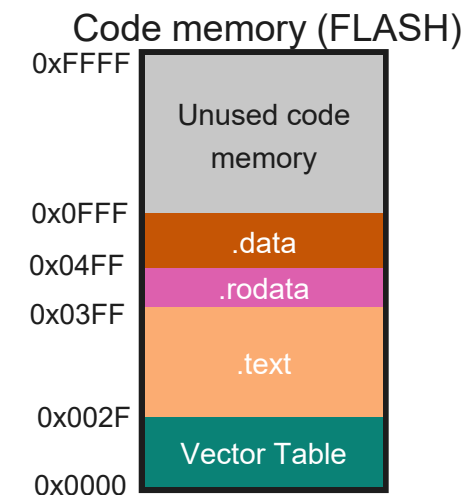
Data	LMA	VMA	Section	Remarks
Global initialized	FLASH	RAM	.data	Startup code copies from FLASH to RAM
Global uninitialized	---	RAM	.bss	Initialized to zero in startup code
Global static initilalized	FLASH	RAM	.data	Startup code copies from FLASH to RAM
Global static uninitialized	---	RAM	.bss	Initialized to zero in startup code
Local initialized	---	STACK (RAM)	---	Consumed at Runtime
Local uninitialized	---	STACK (RAM)	---	Consumed at Runtime
Local static initialized	FLASH	RAM	.data	Startup code copies from FLASH to RAM
Local static uninitialized	---	RAM	.bss	Initialized to zero in startup code
Global const	FLASH	---	.rodata	
Local const	---	STACK (RAM)	---	Like local variable

Source code compilation



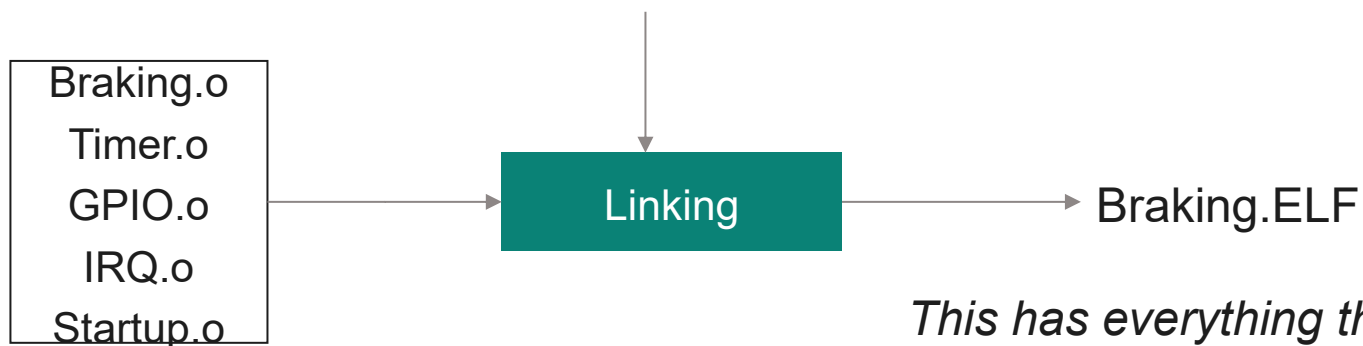
The instructions and data in these object files are more or less ready for flashing.

BUT where must they be stored? What should be their address in memory?



Object linking

Linker script file (Your masterplan ☺)



This has everything that your program loader needs to know!

Before Linking

func1()
func2()
func3()
Data1
Data2



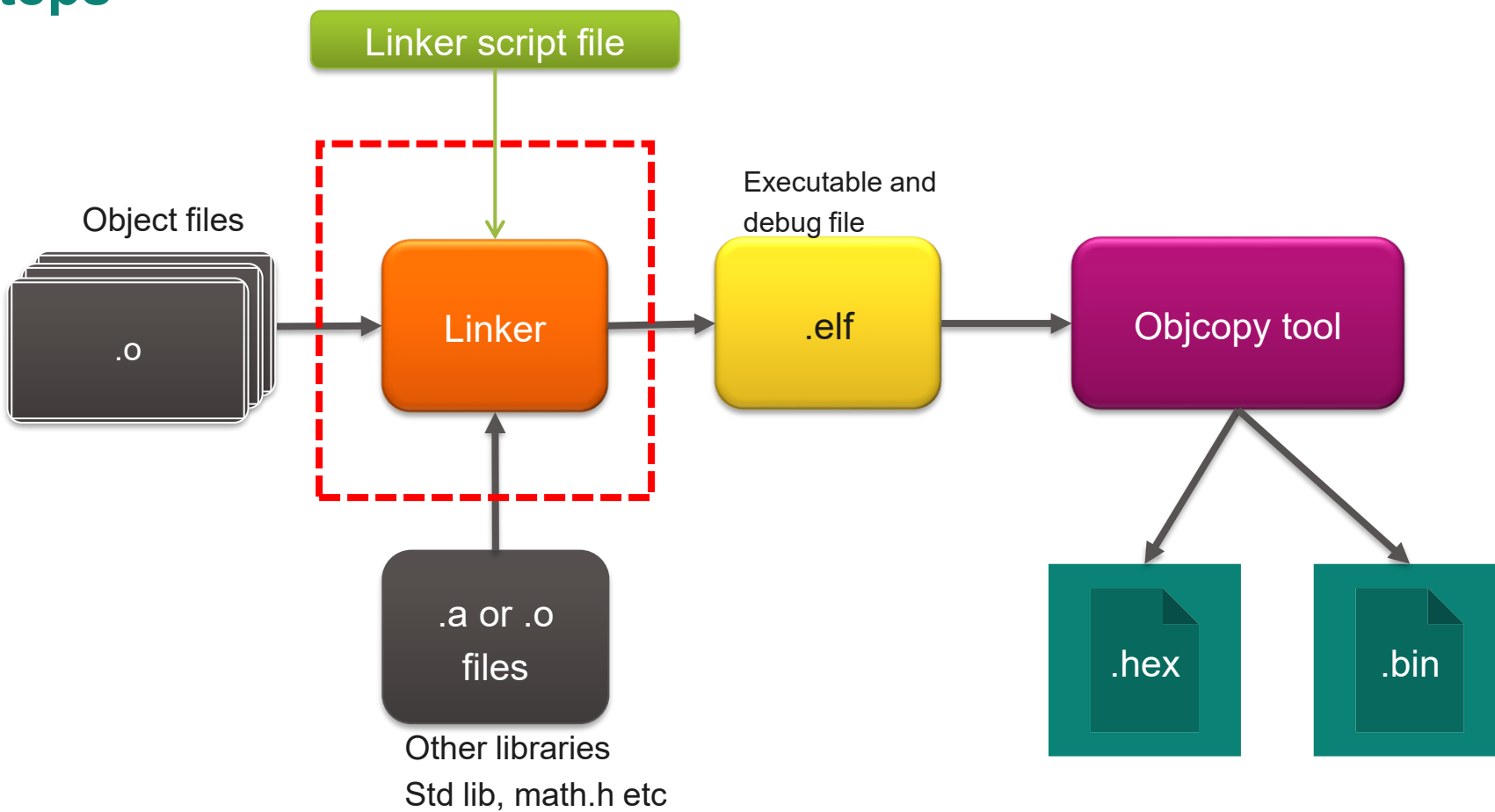
After Linking

func1() 0x20000010 – 0x20001FE
func2() 0x20000200 – 0x200005CB
func3() 0x200005D0 – 0x200005F0
Data1 0x80000000
Data2[8] 0x80000004 – 0x80000023

You specified the range in the linker script!

The linker has only followed your instructions faithfully.

Linker steps



What is an ELF anyways?



This bloke is the ELF!

This is also an ELF! →

Executable and Linkable Format

ELF – Executable File

ELF Header
Segment Header Table
.init
.text
.rodata
.data
.bss
.symtab
.debug
.line
.strtab
Section Header Table

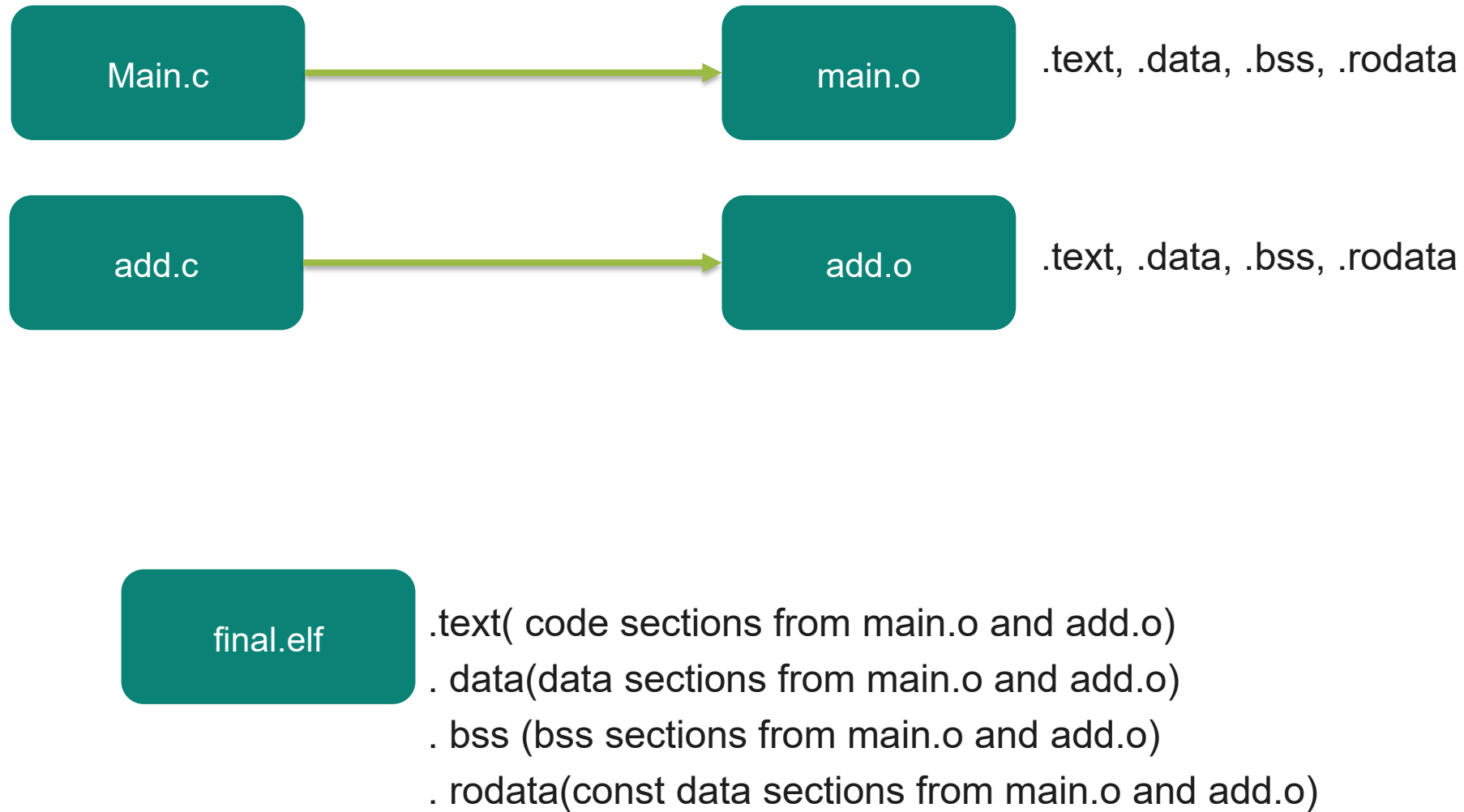
Read-only memory segment (code)

Read-Write memory segment (data)

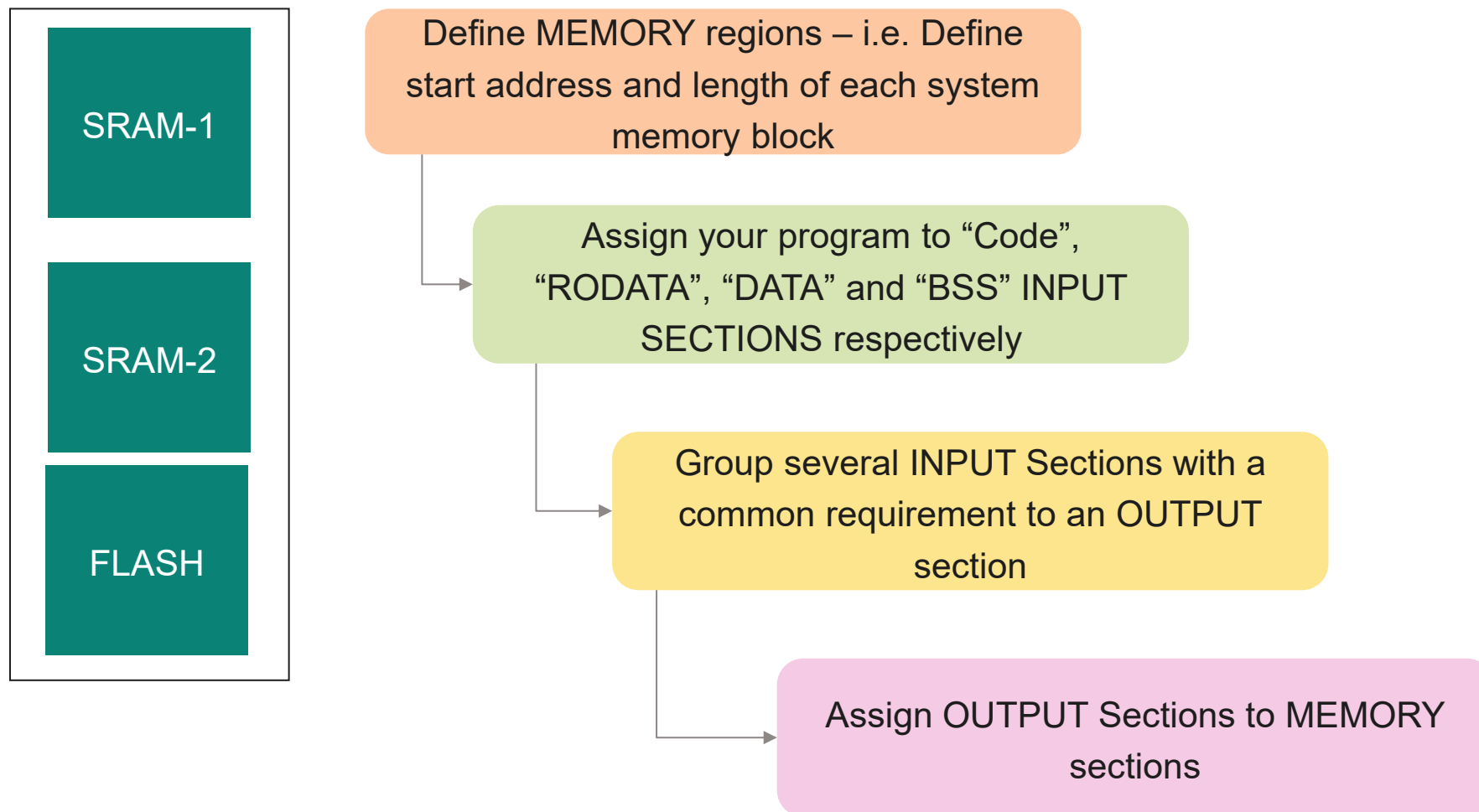
Symbol table & debugging info (Not loaded into memory)

Next few slides deal with ELF creation!

Sections of program in ELF format

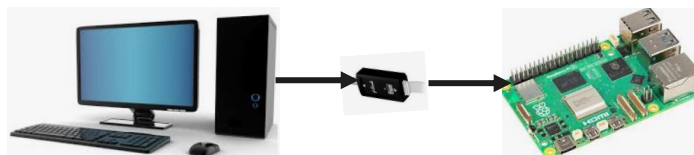


Linker script – Big picture

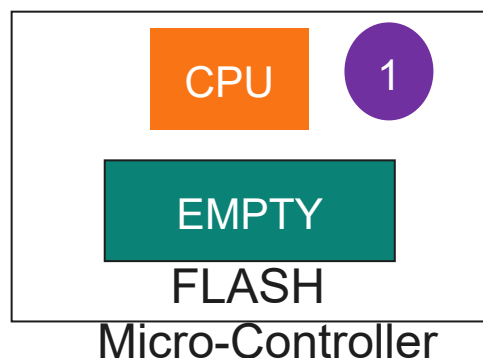


Putting them all together

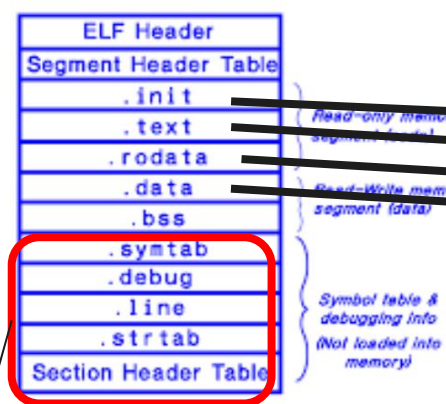
After Flashing your application



Your brand new development board



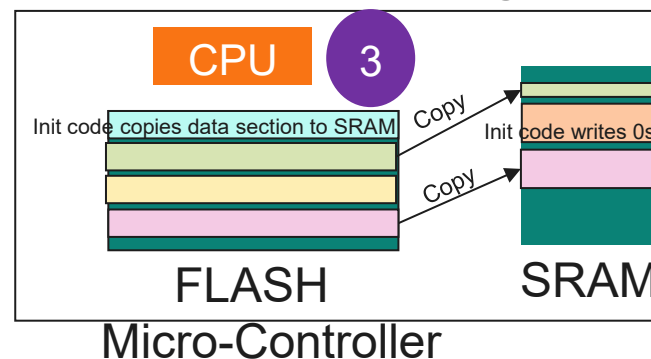
ELF - Executable File



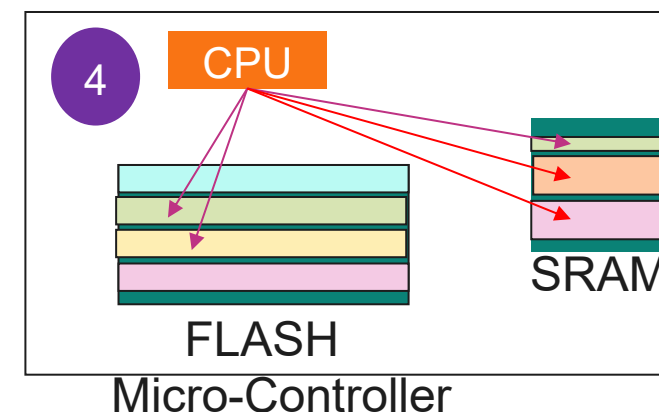
Not flashed.

These are needed only for debugging

At run time (After powering up)

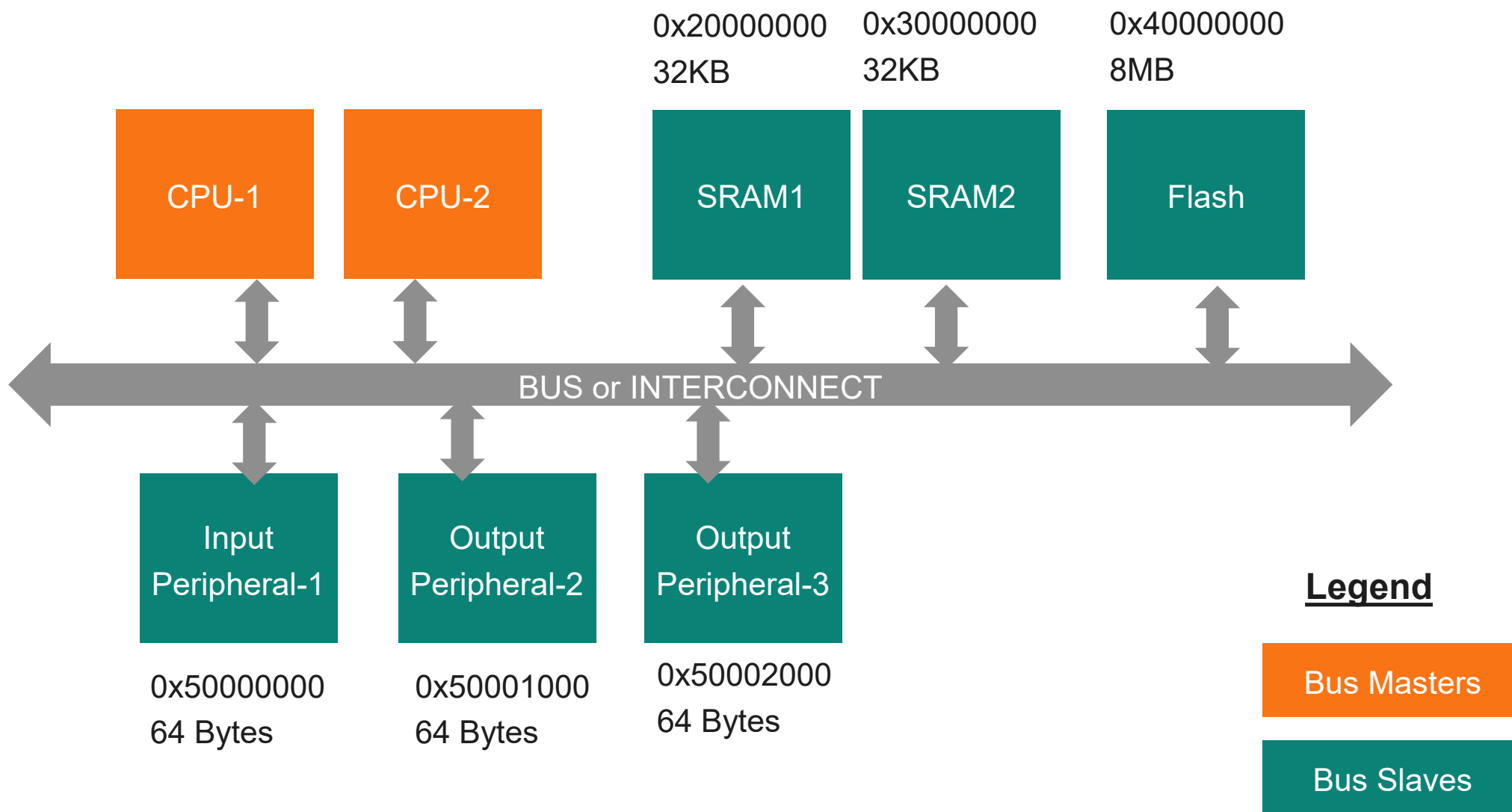


At run time (After program loading)



CPU fetches TEXT and RODATA from flash
CPU accesses DATA and BSS in SRAM
CPU may fetch instructions of time critical TEXT segment which has been copied to SRAM.

An exemplary Microcontroller



Example Linker Script (GNU syntax)

Define MEMORY regions

```
MEMORY
```

```
{
```

```
    name [( attr )] : ORIGIN = origin , LENGTH = len
```

```
...
```

```
}
```

```
MEMORY
```

```
{
```

```
    SRAM1(rwx): ORIGIN: 0x20000000, LENGTH = 32K
```

```
}
```

Example Linker Script (GNU syntax)

Define Sections

SECTIONS

```
{
  Output_Section1:
  {
    Object_File(Input section of the object file)
  } > REGION

  Output_Section2:
  {
    Object_File(Input section of the object file)
  } > REGION
}
```

SECTIONS

```
{
  .fasttext:
  {
    Adc.o(.text)
  } > SRAM1
  .data:
  {
    *(.data)
    *(.bss)
  } > SRAM2
  .text:
  {
    *(.text)
    *(.RODATA)
  } > FLASH
}
```

